DrozyAI

Person 2 Completion Report

Wearable Model & Baseline

Sane Wu

October 10, 2026

Contents

1 Summary **3**

2 What Person 2 Did **3**
2.1 Input . . . 3
2.2 Steps . . . 3

3 The Model and Features **3**
3.1 Features . . . 3
3.2 Model . . . 4
3.3 Why Gradient Boosting . . . 4

4 Results **4**
4.1 Trade-off Curve . . . 4

5 What the Next Person Should Do **5**
5.1 If you are Person 3 (Vision + Fusion + Advisor) . . . 5
5.2 If you are Person 4 (Dashboard + Driver App) . . . 5
5.3 If you want to improve the baseline itself . . . 5

6 What Is Currently on GitHub **6**
6.1 Repository Tree . . . 6
6.2 Data Files on GitHub . . . 6
6.3 Code Files on GitHub . . . 7
6.4 What Is Not on GitHub . . . 7
6.5 Reproducing the Pipeline . . . 7

7 Conclusion **7**

1 Summary

Person 2 of the DrozyAI project is responsible for the **wearable model** that turns smartwatch signals into a drowsiness probability. This report describes:

- what was built in Person 2,
- the model and features used,
- the results achieved,
- what the next person should do,
- and a summary of everything currently on GitHub.

Key Result

The Person 2 deliverable is complete: a gradient-boosting baseline trained on 60-second sliding windows of wearable signals achieves an **AUC of 0.862** under 5-fold stratified cross-validation. Per-window calibrated probabilities are saved to `data/processed/baseline_scores.parquet` for the fusion stage.

2 What Person 2 Did

2.1 Input

Person 2 consumes the per-second aligned table produced by Person 1:

```
data/processed/full_dataset.parquet   (84000 rows x 16 columns)
```

Columns include `subject`, `session`, `second`, `heart_rate`, `temp`, `eda`, `accel_var_x/y/z`, `ibi_mean`, `ibi_count`, `kss`, `drowsy`.

2.2 Steps

1. **Sliding-window feature extraction.** 60-second windows, advanced every 5 seconds. Five features per window.
2. **Model training.** A histogram-based gradient-boosting classifier wrapped in a scikit-learn pipeline with a robust scaler.
3. **Cross-validation.** 5-fold stratified cross-validation, following the original UL-DD paper.
4. **Probability calibration.** Isotonic regression is applied to the raw model probabilities so that a score of 0.7 corresponds to approximately a 70% chance of drowsiness.
5. **Trade-off analysis.** Missed-drowsy rate and false alarms per hour are computed across thresholds from 0.05 to 0.95 and saved as a CSV for the dashboard.

3 The Model and Features

3.1 Features

Five features are computed per 60-second window:

Feature	Source	Description
`hr_mean`	`heart_rate`	Mean heart rate over the window
`rmssd`	`ibi_mean`, `ibi_count`	Root mean square of successive IBI differences (only over seconds with at least one IBI)
`temp_trend`	`temp`	Linear slope of skin temperature over the window
`eda_peaks`	`eda`	Number of local peaks above mean + 1 std
`motion_var`	`accel_var_x/y/z`	Mean of the three accelerometer variance columns

3.2 Model

- **Scaler**: `RobustScaler` (median / IQR).
- **Classifier**: `HistGradientBoostingClassifier` (200 iterations, learning rate 0.05, early stopping).
- **Pipeline**: both wrapped in a single `sklearn.pipeline.Pipeline` so that scaling is fitted only on the training folds.
- **Calibration**: `IsotonicRegression` fitted on the concatenated cross-validation predictions.

3.3 Why Gradient Boosting

- Handles mixed feature scales without manual tuning.
- Robust to outliers thanks to the robust scaler and histogram-based binning.
- Trains in seconds on 11,376 windows with 5 features.
- Provides well-calibrated probabilities after isotonic regression, which is essential for the fusion stage.

4 Results

Key Result

AUC (mean across 5 folds)	**0.862**
AUC standard deviation	0.006
Missed drowsy rate (threshold 0.5)	0.362
False alarms per hour (threshold 0.5)	47.7
Windows used	11,376
Features	5

4.1 Trade-off Curve

Because the same probability threshold controls both missed events and false alarms, the interesting output is the full trade-off curve:

Threshold	Missed drowsy	FA / hour	Caught
0.05	0.006	598.4	0.994
0.10	0.017	526.5	0.983
0.20	0.091	305.3	0.909
0.30	0.159	207.7	0.841
0.40	0.201	166.8	0.799
0.50	0.278	118.1	0.722
0.60	0.349	84.0	0.651
0.70	0.486	44.2	0.514
0.80	0.639	17.9	0.361
0.90	0.806	5.5	0.194
0.95	0.869	2.6	0.131

At threshold 0.6 the system catches about 65% of drowsy windows while producing fewer than 90 false alarms per hour.

5 What the Next Person Should Do

Handoff

Recommended next steps.

5.1 If you are Person 3 (Vision + Fusion + Advisor)

1. Extract eye and mouth features from `data/raw/Extracted_Features/`: Eye Aspect Ratio, PERCLOS, blink rate, blink duration, yawn count.
2. Train a GRU over the last 10 minutes of fused features.
3. Fuse the watch probability (from `baseline_scores.parquet`, column `proba_calibrated`) with the vision probability using late fusion.
4. Wire the fused risk score into the Nemotron 3.5 advisor: a fixed JSON input (current risk, 10-minute trend, top contributing signals) and three possible outputs (*stop now*, *take a break*, *continue*) with a one-sentence reason.
5. Run the latency benchmark with the network off.

5.2 If you are Person 4 (Dashboard + Driver App)

1. Build a Streamlit dashboard that reads `baseline_scores.parquet` and `tradeoff_curve.csv`.
2. Add a threshold slider. When the user moves it, recompute the missed-drowsy rate and false alarms per hour live.
3. Plot the risk timeline per subject with the threshold line.
4. Build the driver app: full-screen traffic light, one-sentence advice, local text-to-speech, and an "I'm alert" button that logs false alarms.

5.3 If you want to improve the baseline itself

- Try a 5-minute window (300 s) with a 30-second step.

- Add skin temperature and EDA trend features over longer horizons.
- Replace the flat classifier with a temporal model (GRU or LSTM) over the last 10 minutes of windows.
- Tune the decision threshold from the trade-off curve rather than using the default 0.5.

6 What Is Currently on GitHub

Note

Data files larger than a few megabytes are not pushed. Only the source code and the three small derived datasets are on GitHub.

6.1 Repository Tree

```
DrozyAI/
|-- .gitignore
|-- README.md
|-- requirements.txt
|-- data/
|   `-- processed/
|       |-- full_dataset.parquet          (1.6 MB)
|       |-- baseline_scores.parquet       (0.5 MB)
|       `-- tradeoff_curve.csv            (small)
|-- docs/
|-- notebooks/
|-- src/
|   |-- __init__.py
|   |-- main.py
|   |-- data/
|   |   |-- __init__.py
|   |   |-- loader.py
|   |   `-- build_dataset.py
|   |-- features/
|   |   |-- __init__.py
|   |   |-- watch_features.py
|   |   |-- video_features.py
|   |   `-- fusion.py
|   |-- models/
|   |   |-- __init__.py
|   |   |-- baseline.py
|   |   |-- gru.py
|   |   `-- calibration.py
|   |-- capture/
|   |-- advisor/
|   |-- dashboard/
|   `-- driver_app/
`-- tests/
```

6.2 Data Files on GitHub

File	Size	Owner
`data/processed/full_dataset.parquet`	1.6 MB	Person 1
`data/processed/baseline_scores.parquet`	0.5 MB	Person 2

File	Size	Owner
`data/processed/tradeoff_curve.csv`	small	Person 2

6.3 Code Files on GitHub

File	Owner
`src/data/loader.py`	Person 1
`src/data/build_dataset.py`	Person 1
`src/features/watch_features.py`	Person 2
`src/models/baseline.py`	Person 2
`src/main.py`	Person 1
Other `src/` files	placeholders for Person 3 and 4

6.4 What Is Not on GitHub

- `data/raw/` — 18 GB of original UL-DD CSVs.
- `data/processed/A_A.parquet` and the other 34 per-session Parquet files.
- `data/processed/window_features.parquet` (509 KB, can be regenerated from `full_dataset.parquet`).
- `reference/` — 95 MB of third-party reference code.
- `.venv/` — local Python virtual environment.

6.5 Reproducing the Pipeline

```
git clone git@github.com:fleurinejaegler/DrozyAI.git
cd DrozyAI
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# Option A: use the shared aligned dataset
python -m src.models.baseline

# Option B: rebuild from raw data (requires 18 GB of UL-DD)
python -m src.main
python -m src.features.watch_features
python -m src.models.baseline
```

7 Conclusion

Person 2 delivered a complete wearable baseline with an AUC of 0.862, calibrated per-window probabilities, and a full trade-off curve. The outputs are ready to be consumed by Person 3 (fusion and advisor) and Person 4 (dashboard and driver app). All source code and the three small derived datasets are on GitHub; the raw and per-session data remain local.